

Segmentação de Áudio por Furo

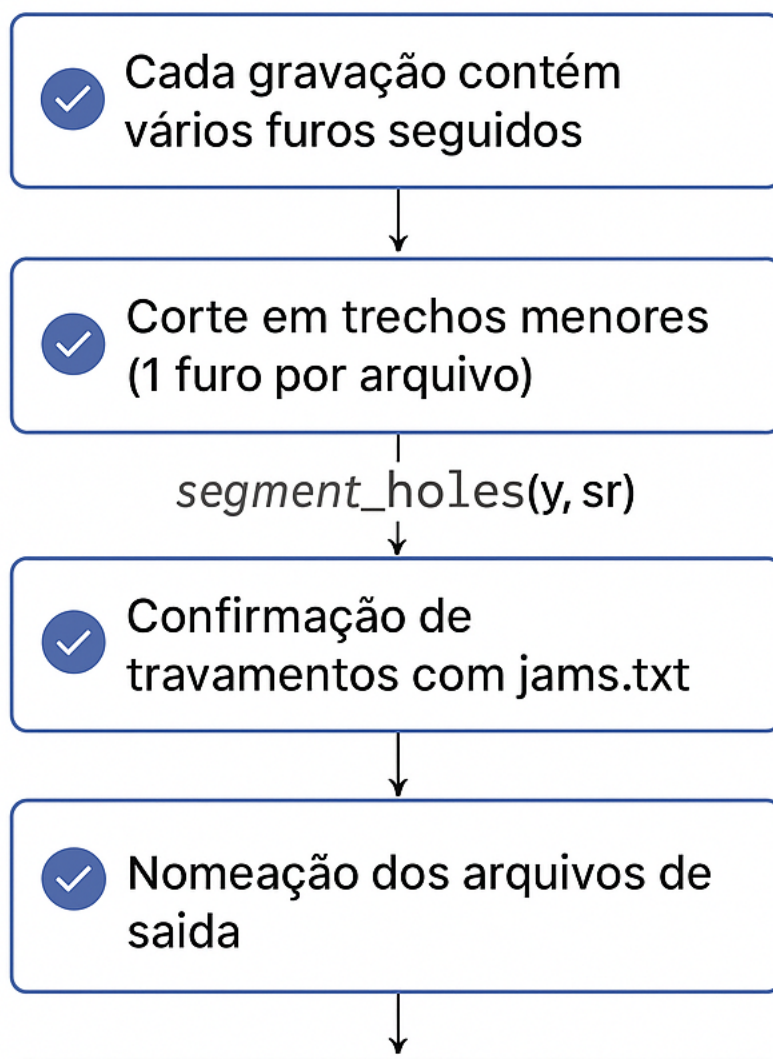
Este documento descreve o funcionamento atualizado do módulo `segment.py`, responsável por **segmentar gravações de perfuração em trechos individuais (furos)** com base na **energia do sinal (RMS)** e no controle de travamentos via `jams.txt`.

✓ Resumo Geral

O módulo:

- Segmenta gravações de áudio contínuas em **furos individuais**.
- Usa **energia RMS** para detectar **início e fim de cada furo**.
- Confirma **furos travados** com base no arquivo `jams.txt`.
- **Ignora automaticamente** arquivos ocultos gerados pelo macOS (`._arquivo.wav`).
- Nomeia e salva os segmentos com **padrão consistente e informativo**.
- Gera **metadados completos** e uma **overview visual por drill**.

Segmentação de Áudio por Furo



- Regularidade dos furos

Componentes Extras

```
load_datalogger()  
metadata_list.append(**}  
holes_before_fail  
pd.DataFrame(...).to_csv(...)
```



Segmentos
de áudio

Passo a Passo da Lógica

1. Processamento de gravações

O script percorre `data/standardized/{drill_id}/`, assumindo que cada `.wav` contém vários furos consecutivos de um mesmo microfone.

Durante o loop principal, arquivos ocultos como `._arquivo.wav` são ignorados para evitar erros no macOS.

2. Segmentação em furos

A função `segment_holes(y, sr)` identifica os limites de cada furo com base na energia RMS:

```
rms = librosa.feature.rms(y=y, frame_length=2048, hop_length=1024)[0]  
rms = rms / np.max(rms)  
mask = rms > ENERGY_THRESHOLD
```

- Define regiões com energia acima do limiar (`ENERGY_THRESHOLD`).
- Cada bloco contínuo é um **furo**.
- Ignora durações curtas (`MIN_HOLE_DURATION` = 0.5 s).
- Retorna intervalos (`start, end`) em amostras.

Esses trechos são extraídos e salvos:

```
sf.write(out_path, y[start:end], sr)
```

3. Confirmação de travamentos com `jams.txt`

O arquivo `jams.txt` é lido da pasta correspondente em `data/raw/{drill_id}/`:

```
jams = load_jams(raw_drill_folder)
fail = 1 if idx_local in jams else 0
```

- Os índices de furos com travamento são definidos manualmente em `jams.txt` (ex.: 39, 10, 3).
- Furos marcados como travados recebem o sufixo `_jam` no nome do arquivo.

4. Nomeação dos arquivos segmentados

```
filename = f"{drill_id}_hole{idx_local:02d}_{mic_type}_{mic_id}_{position}"
if fail:
    filename += "_jam"
filename += ".wav"
```

Exemplos:

```
drill01_hole05_ult_5_int.wav
drill01_hole06_ult_5_int_jam.wav
```

5. Identificação automática de microfones

O script infere metadados do nome do arquivo:

Campo	Regra de detecção
<code>mic_type</code>	contém "ult" ou "ultrasonic" → ult, senão com
<code>mic_id</code>	dígitos no nome do arquivo
<code>position</code>	ext se <code>mic_id</code> ∈ [1, 2, 3], senão int

6. Regularidade dos furos

Os furos costumam ter **duração regular**. Travamentos produzem padrões irregulares de energia — o algoritmo já detecta e marca automaticamente com `_jam`.

Funcionalidades Extras

Parte	Função	Impacto
<code>gerar_overview_drill()</code>	Cria gráfico "overview" com furos e falhas	Visualização rápida
<code>metadata_list.append({...})</code>	Registra informações de cada furo	CSV consolidado
<code>pd.DataFrame(...).to_csv(...)</code>	Gera <code>segmented_metadata.csv</code>	Análise posterior

Parte	Função	Impacto
Filtro <code>._</code>	Ignora arquivos ocultos do macOS	Estabilidade extra
<code>warnings.filterwarnings(...)</code>	Suprime alertas de depreciação do librosa	Limpeza de logs

Estrutura Esperada de Pastas

```
data/
├── raw/
│   ├── drill03/
│   │   ├── jams.txt
│   │   └── column_1/
│   │       └── datalogger/
│   │           ├── voltage.csv
│   │           └── current.csv
│   └── standardized/
│       ├── 03/
│       │   ├── 03_common_3_ext.wav
│       │   ├── 03_ultrasonic_5_int.wav
│       │   └── ...
│       └── segmented/
│           ├── 03/
│           │   ├── 03_hole01_com_3_ext.wav
│           │   ├── 03_hole02_com_3_ext.wav
│           │   └── 03_hole05_ult_5_int_jam.wav
```

Fluxo do Processo

```
1 Itera pastas em data/standardized/
2 Define drill_id = nome da pasta
3 Localiza raw correspondente (por número)
4 Lê jams.txt -> furos travados
5 Ignora arquivos ocultos (._)
6 Extrai mic_type, mic_id e position
7 Carrega áudio (librosa)
8 Calcula energia RMS e segmenta furos
9 Corta e salva trechos (_jam se aplicável)
10 Gera CSV de metadados e overview visual
```

Resultado Final

• Áudios segmentados:

```
data/segmented/03/03_hole01_com_3_ext.wav
data/segmented/03/03_hole05_ult_5_int_jam.wav
```

- Metadados (segmented_metadata.csv):

drill_id	hole_idx	mic_type	mic_id	position	fail	filename	start_sample	end_sample
----------	----------	----------	--------	----------	------	----------	--------------	------------



Conclusão

O módulo atualizado de segmentação:

- Segmenta gravações de forma **robusta e automática**.
- Identifica **furos travados** com base no **jams.txt**.
- **Ignora arquivos invisíveis** do macOS, evitando falhas de leitura.
- Gera arquivos **padronizados e confiáveis** para análise posterior.
- Produz uma **overview visual** para rápida verificação de falhas.

💡 *O pipeline está pronto para integração com análises de datalogger e correlação entre energia e parâmetros de perfuração.*